

CricTactix-AI: Backend Architecture Deep Dive

An Interview-Level Engineering & System Design Guide

This document provides a highly detailed, comprehensive, and production-grade breakdown of the **CricTactix-AI Backend Application**. It is designed to give engineers, developers, and technical interview candidates an in-depth understanding of the system's architecture, design decisions, data ingestion pipelines, database schemas, and AI agent reasoning loops.

■ Table of Contents

- 1. System Architecture & Core Philosophy
- 2. Codebase Directory Structure
- 3. The Six-Model Relational Schema
- 4. High-Performance Ingestion Pipeline (**CODE PLACEHOLDER 0**)
- 5. The REST API & Scorecard Generation Service
- 6. The AI Analytics Agent (**CODE PLACEHOLDER 0**)
- 7. End-to-End Request Lifecycle
- 8. Production Scaling: SQLite to PostgreSQL
- 9. Interview Preparation Guide (15+ Q&As)

1. System Architecture & Core Philosophy

CricTactix-AI is built on a **decoupled, API-first architecture** consisting of a Next.js frontend and a high-performance **FastAPI** backend.



```
Backend/
■■■ main.py                # Entrypoint; registers all API routers, middleware & CORS
■■■ requirements.txt       # Python dependency definitions
■■■ cricket_ai.db         # The active SQLite relational database (gitignored)
■■■ test_cricket_ai.db    # Isolated database used specifically during Pytest runs
■
■■■ app/
■   ■■■ __init__.py
■   ■
■   ■■■ core/             # Centralized infrastructure and setup
```

```

■ ■ ■■■ config.py           # Config manager (Pydantic BaseSettings, env resolver)
■ ■ ■■■ database.py         # SQLAlchemy engine setup and thread-safe session generator
■ ■ ■■■ helpers.py         # String matching & alias dictionaries (Teams & Venues)
■ ■
■ ■■■ models/              # Database Schema Definition (SQLAlchemy Declarative Models)
■ ■ ■■■ match.py           # Match metadata and canonical team mapping
■ ■ ■■■ delivery.py        # Ball-by-ball delivery information (micro-level data)
■ ■ ■■■ player.py          # Player profile definition
■ ■ ■■■ team.py            # Team identities & alias structures
■ ■ ■■■ team_season.py     # Tracks team participations across calendar years
■ ■ ■■■ team_season_player.py # Roster mapping of players to a specific team's season
■ ■
■ ■■■ schemas/            # Request/Response DTO Validation contracts (Pydantic)
■ ■ ■■■ chat.py           # Contracts for chat-related endpoints
■ ■ ■■■ matches.py        # Contracts for match listings, scorecards and stats
■ ■
■ ■■■ services/           # Reusable business logic (domain service layer)
■ ■ ■■■ __init__.py       #
■ ■ ■■■ match_service.py  # Scorecard builder & aggregated statistical calculations
■ ■
■ ■■■ api/                # Endpoint routers grouped by domain
■ ■ ■■■ routes/
■ ■ ■■■   ■■■ chat.py     # POST /api/chat - LangChain-based SQL Agent endpoint
■ ■ ■■■   ■■■ health.py  # GET /api/health - Liveness / readiness check
■ ■ ■■■   ■■■ matches.py # GET /api/matches, stats, and background db seeding
■ ■
■ ■■■ data_pipeline/      # Raw data ingestion utilities
■ ■ ■■■ load_cricsheet.py # Bulk ingestion, JSON parser & phase calculator
■
■■■ tests/                # Automated Test Suite (Pytest framework)
■ ■ ■■■ conftest.py       # Setup and tear down fixtures for test DB
■ ■ ■■■ test_api.py       # Integration test cases for API routes

```

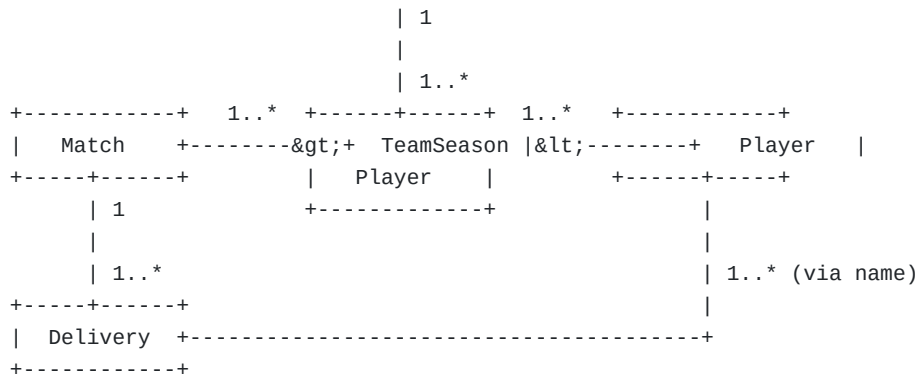
3. The Six-Model Relational Schema

The database utilizes six declarative SQLAlchemy models. Unlike simple flattened schemas, this structure maps players, rosters, seasons, matches, and delivery metrics in a fully normalized relational graph.

```

+-----+
|   Team   |
+-----+
      | 1
      |
      | 1..*
+-----+
| TeamSeason |
+-----+

```



1. **CODE_PLACEHOLDER_0 (CODE_PLACEHOLDER_1 table)**

Stores canonical records of franchises and national teams.

- **CODE_PLACEHOLDER_0** (Integer, Primary Key)
- **CODE_PLACEHOLDER_0** (String, Unique, Index): Canonical team name (e.g., **CODE_PLACEHOLDER_1**).
- **CODE_PLACEHOLDER_0** (String, Nullable): Abbreviated tag (e.g., **CODE_PLACEHOLDER_1**).
- **CODE_PLACEHOLDER_0** (JSON, Nullable): Dynamic list of recognized variations.

2. **CODE_PLACEHOLDER_0 (CODE_PLACEHOLDER_1 table)**

Stores canonical player profiles.

- **CODE_PLACEHOLDER_0** (Integer, Primary Key)
- **CODE_PLACEHOLDER_0** (String, Unique, Index): Full player name (e.g., **CODE_PLACEHOLDER_1**).
- **CODE_PLACEHOLDER_0** (String, Nullable): e.g., **CODE_PLACEHOLDER_1**.
- **CODE_PLACEHOLDER_0** (JSON, Nullable): Holds extensible metadata (batting style, birthdate).

3. **CODE_PLACEHOLDER_0 (CODE_PLACEHOLDER_1 table)**

Bridges a **CODE_PLACEHOLDER_0** to a calendar year.

- **CODE_PLACEHOLDER_0** (Integer, Primary Key)
- **CODE_PLACEHOLDER_0** (Integer, Foreign Key pointing to **CODE_PLACEHOLDER_1**)
- **CODE_PLACEHOLDER_0** (Integer, Index): Calendar year of active competition.
- **CODE_PLACEHOLDER_0** (String, Nullable): League classification (e.g., **CODE_PLACEHOLDER_1**).

4. **CODE_PLACEHOLDER_0 (CODE_PLACEHOLDER_1 table)**

Defines team rosters and roles for a specific year.

- **CODE_PLACEHOLDER_0** (Integer, Primary Key)
- **CODE_PLACEHOLDER_0** (Integer, Foreign Key pointing to **CODE_PLACEHOLDER_1**)
- **CODE_PLACEHOLDER_0** (Integer, Foreign Key pointing to **CODE_PLACEHOLDER_1**)
- **CODE_PLACEHOLDER_0** (String, Nullable): Playing role (e.g., **CODE_PLACEHOLDER_1**, **CODE_PLACEHOLDER_2**, **CODE_PLACEHOLDER_3**).
- **CODE_PLACEHOLDER_0** (Integer, Nullable): Shirt number.

5. **CODE_PLACEHOLDER_0 (CODE_PLACEHOLDER_1 table)**

Represents a match event with references to both teams.

- **CODE_PLACEHOLDER_0** (Integer, Primary Key)
- **CODE_PLACEHOLDER_0** (String, Unique, Index): Unique match code from Cricsheet.
- **CODE_PLACEHOLDER_0** (String): e.g., **CODE_PLACEHOLDER_1**.
- **CODE_PLACEHOLDER_0** (String), **CODE_PLACEHOLDER_1** (String): Location data.
- **CODE_PLACEHOLDER_0** (Date): Play commencement date.
- **CODE_PLACEHOLDER_0** (String), **CODE_PLACEHOLDER_1** (String): Flat team names for fast query filters.
- **CODE_PLACEHOLDER_0** (String): Winner team name.
- **CODE_PLACEHOLDER_0** / **CODE_PLACEHOLDER_1** (Integer, Foreign Keys to **CODE_PLACEHOLDER_2**): Canonical team relationships.

6. **CODE_PLACEHOLDER_0 (CODE_PLACEHOLDER_1 table)**

The micro-data table storing ball-by-ball actions.

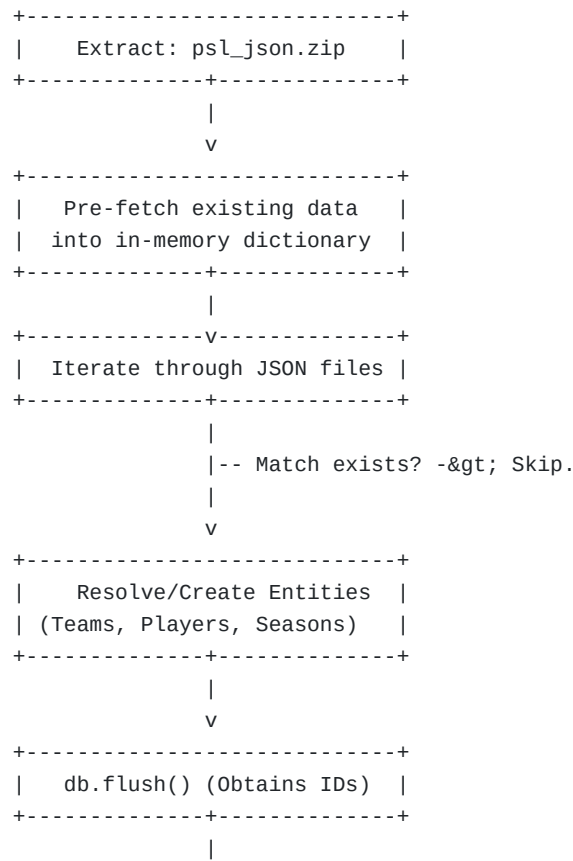
- **CODE_PLACEHOLDER_0** (Integer, Primary Key)
- **CODE_PLACEHOLDER_0** (Integer, Foreign Key pointing to **CODE_PLACEHOLDER_1**)
- **CODE_PLACEHOLDER_0** (Integer): Innings indicator (1 or 2).
- **CODE_PLACEHOLDER_0** (Integer), **CODE_PLACEHOLDER_1** (Integer): State tracking.
- **CODE_PLACEHOLDER_0** (String), **CODE_PLACEHOLDER_1** (String): Flat team names.
- **CODE_PLACEHOLDER_0** (String), **CODE_PLACEHOLDER_1** (String), **CODE_PLACEHOLDER_2** (String): Player references.
- **CODE_PLACEHOLDER_0** (Integer): Runs scored directly from the bat.

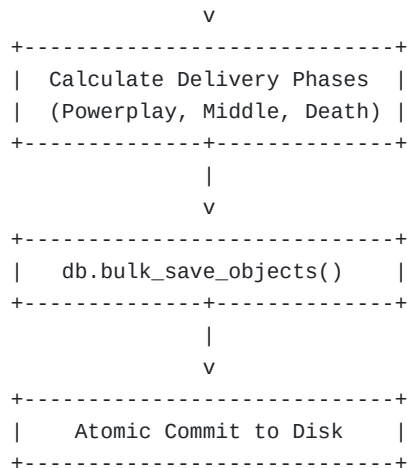
- **CODE_PLACEHOLDER_0** (Integer): Extra runs (wides, no-balls, byes, leg-byes).
 - **CODE_PLACEHOLDER_0** (Integer): Total runs scored on the delivery (**CODE_PLACEHOLDER_1**).
 - **CODE_PLACEHOLDER_0** (String, Nullable): Dismissal method (e.g., **CODE_PLACEHOLDER_1**, **CODE_PLACEHOLDER_2**, **CODE_PLACEHOLDER_3**).
 - **CODE_PLACEHOLDER_0** (String, Nullable): Name of dismissed player.
 - **CODE_PLACEHOLDER_0** (String, Nullable): Powerplay, Middle, or Death.
-

4. High-Performance Ingestion Pipeline (CODE_PLACEHOLDER_0)

A primary bottleneck in data-heavy analytics apps is database insertion speed. A single T20 match contains ~250 deliveries. Loading 100 matches flatly would require 25,000+ DB transactions, slowing down seeding.

To overcome this, CricTactix-AI uses a highly optimized, custom ETL pipeline in **CODE_PLACEHOLDER_0**:





Ingestion Optimizations Explained

1. In-Memory Entity Caching:

Before parsing a single JSON match file, the script queries the database *once* to pull all existing **CODE_PLACEHOLDER_0**, **CODE_PLACEHOLDER_1**, **CODE_PLACEHOLDER_2**, and **CODE_PLACEHOLDER_3** entities into Python dictionaries:

```
existing_teams = {t.name: t for t in db.query(Team).all()}
existing_players = {p.name: p for p in db.query(Player).all()}
```

When processing matches, the pipeline checks these dictionaries first. This completely avoids executing thousands of redundant **CODE_PLACEHOLDER_0** queries (avoiding the \$N+1\$ query problem during ingestion).

2. The **CODE_PLACEHOLDER_0** Technique:

To map relationships during a single transaction, the script uses SQLAlchemy's **CODE_PLACEHOLDER_0** rather than **CODE_PLACEHOLDER_1**. Calling **CODE_PLACEHOLDER_2** sends SQL instructions to the database, allowing SQLite to generate primary key IDs (e.g., **CODE_PLACEHOLDER_3**) in memory without executing expensive disk I/O operations.

3. Over-based Phase Calculation:

During delivery parsing, the pipeline calculates match phases dynamically depending on the current over:

```
def get_phase(over: int, match_type: str) -> str:
    if match_type and "T20" in match_type.upper():
        if over < 6: return "Powerplay"
        elif over < 15: return "Middle"
        else: return "Death"
```

```
return "Unknown"
```

4. SQLAlchemy **CODE_PLACEHOLDER_0**:

Instead of using standard **CODE_PLACEHOLDER_0** inside loops, all **CODE_PLACEHOLDER_1** instances for a match are appended to a list and pushed in a single batch insert operation:

```
db.bulk_save_objects(deliveries_to_insert)
```

5. Atomic Commit:

Only after all match files have been completely read and flushed does the script issue a single, atomic **CODE_PLACEHOLDER_0** call. This ensures either all data is successfully written, or a total rollback occurs if an error is thrown, maintaining transaction safety.

5. The REST API & Scorecard Generation Service

The API layer is structured to support flexible searching and complex aggregations.

Dynamic Match Listing & Bidirectional Team Filtering

The **CODE_PLACEHOLDER_0** route includes standard features such as pagination, text searching, and bidirectional team filters:

- **Fuzzy Name and Venue Aliasing:** Integrates **CODE_PLACEHOLDER_0** and **CODE_PLACEHOLDER_1** from **CODE_PLACEHOLDER_2**. If a user queries **CODE_PLACEHOLDER_3**, the system automatically queries variants like **CODE_PLACEHOLDER_4**.
- **Bidirectional Match Filters:** If both **CODE_PLACEHOLDER_0** and **CODE_PLACEHOLDER_1** are provided, the system queries for match occurrences where:

$$((\text{Team 1} = A \wedge \text{Team 2} = B) \vee (\text{Team 1} = B \wedge \text{Team 2} = A))$$

Scorecard Aggregation Pipeline

Rather than saving structured scorecard reports directly, the application aggregates flat ball-by-ball **CODE_PLACEHOLDER_0** in the database into standard scorecards dynamically in **CODE_PLACEHOLDER_1**:

```
def build_scorecard_from_deliveries(deliveries: List[Delivery]) -> List[Dict[str, Any]]:
```


- # 1. Maps each innings (1st innings vs 2nd innings)
- # 2. Accumulates runs_total and extras_total for the team
- # 3. Tracks batsmen: runs scored, balls faced (excluding Wides), strike rate calculation
- # 4. Tracks bowlers: runs conceded, balls bowled, economy rate, and wickets

Bowling Wicket Calculation

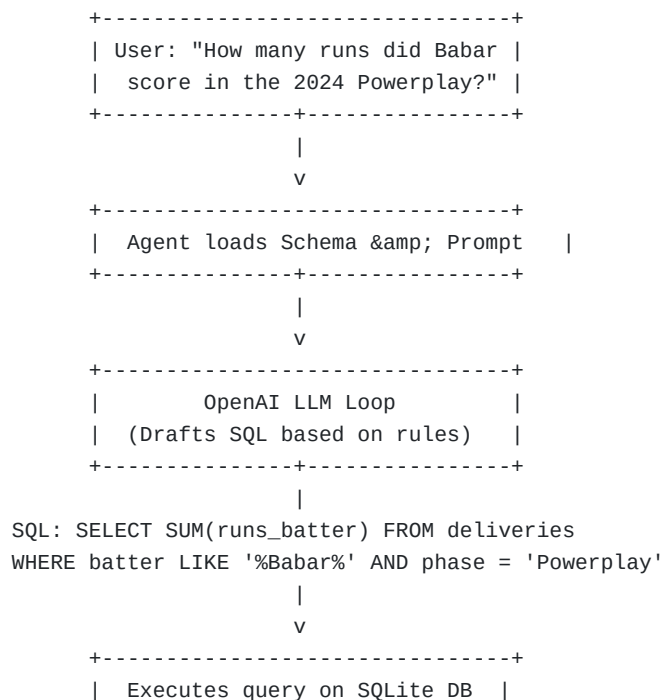
To match ICC scoring rules, the system only counts bowler wickets based on active dismissals:

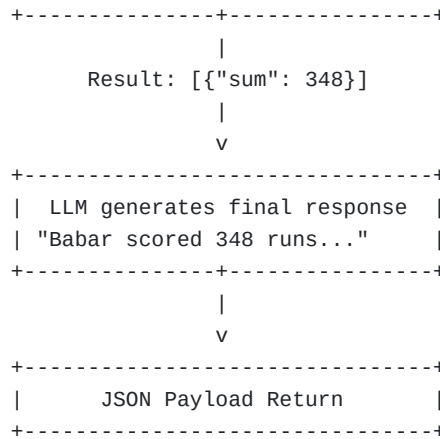
```
bowler_wickets = ('bowled', 'caught', 'caught and bowled', 'lbw', 'stumped', 'hit wicket')
if delivery.wicket_type in bowler_wickets:
    bowler_stats["wickets"] += 1
```

Note: Run outs, retired hurt, retired out, and obstructing the field are correctly ignored from bowler statistics.

6. The AI Analytics Agent (CODE_PLACEHOLDER_0)

The most advanced route is **CODE_PLACEHOLDER_0**, which hosts a **LangChain SQL Database Agent**. It acts as a natural language compiler that translates plain English queries into structured SQLite queries, runs them, and interprets the results.





Agent Configuration Details

- **Engine:** ChatOpenAI (`CODE_PLACEHOLDER_0`).
- **Temperature = 0:** Minimizes hallucinations and ensures deterministic responses.
- **Safety Prompts:** A rigid system instruction (`CODE_PLACEHOLDER_0` and `CODE_PLACEHOLDER_1`) defines available tables and dictates how to calculate statistics.

The AI Safety Prompt Guardrails:

1. **Bowler Wickets constraint:** Explicitly defines bowler wickets to ensure the agent uses the correct conditional `CODE_PLACEHOLDER_0` query instead of a generic `CODE_PLACEHOLDER_1` of dismissals.
2. **Name Matching rule:** Dictates that the AI must always use case-insensitive SQL `CODE_PLACEHOLDER_0` clauses (e.g., `CODE_PLACEHOLDER_1`) instead of strict matches (e.g., `CODE_PLACEHOLDER_2`), preventing failures due to minor spelling differences.
3. **Information Grounding rule:** Forbids the agent from fabricating data. If a requested player or stat is missing, it must return a clear refusal.

7. End-to-End Request Lifecycle

Here is the exact journey of a request sent to the `CODE_PLACEHOLDER_0` route:

1. **HTTP Handshake:** The frontend issues an HTTP GET request to `CODE_PLACEHOLDER_0`.
2. **CORS Validation:** FastAPI's `CODE_PLACEHOLDER_0` intercepts the request, inspecting origins, methods, and headers, and allows access since `CODE_PLACEHOLDER_1` is trusted.

3. **Schema Validation:** Pydantic validates incoming URL query parameters against type constraints (e.g., checking that **CODE_PLACEHOLDER_0** is an integer ≥ 1).

4. **Session Allocation:** FastAPI retrieves a thread-safe database connection session from the **CODE_PLACEHOLDER_0** generator:

```
def get_db():
    db = SessionLocal()
    try: yield db
    finally: db.close()
```

5. **Fuzzy Expansion:** The controller maps the keyword **CODE_PLACEHOLDER_0** through **CODE_PLACEHOLDER_1**, expanding it to match **CODE_PLACEHOLDER_2**.

6. **SQL Generation & Execution:** SQLAlchemy generates the corresponding SQL query with dynamic parameters and sends it to SQLite:

```
SELECT * FROM matches
WHERE team_1 LIKE '%Karachi%' OR team_2 LIKE '%Karachi%'
ORDER BY start_date DESC
LIMIT 50 OFFSET 0;
```

7. **Response Serialization:** The fetched match models are serialized into a clean JSON structure, the database session is closed, and the server returns a **CODE_PLACEHOLDER_0** JSON response.

8. Production Scaling: SQLite to PostgreSQL

SQLite is excellent for development because it is a lightweight, zero-configuration file on disk. However, in production environments with high traffic, it suffers from write locks and concurrency limitations.

How to Scale the App to PostgreSQL:

SQLAlchemy makes database migration straightforward. To switch databases:

1. Install Postgres Driver:

```
pip install psycopg2-binary
```

2. Update Environment Variable:

Change the **CODE_PLACEHOLDER_0** in **CODE_PLACEHOLDER_1** (or production environment settings):

```
DATABASE_URL=postgresql://db_user:db_password@localhost:5432/crictactix_db
```

3. Adjust Connection Configurations:

PostgreSQL connection engines handle threading differently than SQLite. The application handles this dynamically in **CODE_PLACEHOLDER_0**:

```
# SQLite requires check_same_thread=False, PostgreSQL does not
connect_args = {"check_same_thread": False} if "sqlite" in settings.DATABASE_URL else {}
engine = create_engine(settings.DATABASE_URL, connect_args=connect_args)
```

4. Run Background Seeding:

Call the **CODE_PLACEHOLDER_0** endpoint. The pipeline will automatically create the PostgreSQL tables using **CODE_PLACEHOLDER_1** and seed the data without needing any SQL schema adjustments.

9. Interview Preparation Guide (15+ Q&As)

These interview questions are designed to prepare you for senior backend and AI system design interviews:

Q1: Why did you choose FastAPI over traditional frameworks like Django or Flask?

Answer:

Django is an all-in-one framework. However, its built-in ORM is heavy, it lacks async-native performance out of the box, and it includes unnecessary features for a decoupled single-page application.

Flask is lightweight but lacks native async handling, requires manual integration for type safety, and does not build API schemas automatically.

FastAPI is asynchronous-native, built on **Starlette** and **Pydantic** for high performance. It generates automatic Swagger docs using the OpenAPI standard, and provides a powerful dependency injection system (**CODE_PLACEHOLDER_0**), making it perfect for modern API development and LLM integration.

Q2: What is the \$N+1\$ query problem, and how did you prevent it during database seeding?

Answer: The \$N+1\$ query problem occurs when an application executes one query to fetch parent records, and then issues \$N\$ subsequent queries to fetch child records for each parent. During parsing, looking up players or teams in a loop would cause thousands of database hits.

We solved this by **pre-fetching all existing records into memory** before processing:

```
existing_players = {p.name: p for p in db.query(Player).all()}
```

We then perform fast dictionary lookups in Python ($O(1)$ complexity) instead of querying the database, reducing database interaction to a single initial query.

Q3: How does the AI agent translate natural language into SQL safely? Doesn't it pose a risk of SQL Injection?

Answer: There are two layers of safety protecting the database:

1. **Parameterized Queries:** Under the hood, LangChain's SQLDatabase utility uses SQLAlchemy to connect to the database. All queries are parsed and executed using parameterized execution, meaning user inputs are treated as strings rather than executable SQL commands.
2. **Read-Only / Sandboxed Scope:** In a production environment, the database user credentials passed to the AI agent should only have read access (**CODE_PLACEHOLDER_0** permissions). The agent is physically unable to run write operations like **CODE_PLACEHOLDER_1** or **CODE_PLACEHOLDER_2** because the underlying database account lacks the privileges to execute them.

Q4: Why did you use **CODE_PLACEHOLDER_0** instead of **CODE_PLACEHOLDER_1** inside the match processing loops during data ingestion?

Answer: A **CODE_PLACEHOLDER_0** commits the current transaction to disk, writing all changes permanently. This requires slow disk I/O operations.

CODE_PLACEHOLDER_0 sends the pending operations to the database memory buffer instead. This allows the database to check constraints and generate primary keys (so we can get **CODE_PLACEHOLDER_1** to link deliveries) without writing anything to disk. By using **CODE_PLACEHOLDER_2** in our loops and calling **CODE_PLACEHOLDER_3** once at the very end, we perform a single, fast disk write transaction.

Q5: How do you handle database sessions in FastAPI routes? What happens if a database query fails midway?

Answer: We use FastAPI's dependency injection system with a generator function **CODE_PLACEHOLDER_0**:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

FastAPI injects a session into the route and ensures that the session is closed when the request finishes, even if an exception is thrown. For write operations in our pipeline, we wrap the logic in a **CODE_PLACEHOLDER_0** block and call **CODE_PLACEHOLDER_1** on failure to revert any partial changes, keeping our database consistent.

Q6: Why did you set the temperature of the ChatOpenAI model to **CODE_PLACEHOLDER_0** in the chat router?

Answer: Temperature controls the randomness of an LLM's responses. A higher temperature makes the output more creative but unpredictable.

For database query translation, we need the output to be deterministic and reliable. By setting the temperature to **CODE_PLACEHOLDER_0**, we force the LLM to choose the most likely token every time, ensuring it generates the exact same SQL query for the same question every run.

Q7: If your database grows to millions of deliveries, how would you optimize the scorecard generation?

Answer: Dynamically aggregating millions of deliveries on every request will eventually slow down. To scale this, we could use **Materialized Views** or a **Cached Read-Model**. We would pre-calculate and store the final match scorecard JSON in a column on the **CODE_PLACEHOLDER_0** table (e.g. **CODE_PLACEHOLDER_1**) during the ingestion pipeline. Since match results are static and never change, this allows us to serve scorecards instantly using a single **CODE_PLACEHOLDER_2** query, completely eliminating delivery aggregation during API calls.

Q8: What is the purpose of Pydantic schemas in this application, and how do they differ from SQLAlchemy models?

Answer: This represents a clean **separation of concerns**:

- **SQLAlchemy Models** define our database tables, constraints, types, and database-level relationships.
- **Pydantic Schemas** define our API contracts (data validation and serialization models).

For example, our **CODE_PLACEHOLDER_0** table has 10 columns, but our API request may only require 3 fields. Pydantic handles request validation and formats the database models into clean JSON responses, ensuring database implementation details are kept separate from the public API.

Q9: How does the AI agent know how to construct correct queries for complex metrics like bowler wickets or batsman runs?

Answer: We guide the agent using strict instructions in **CODE_PLACEHOLDER_0** and **CODE_PLACEHOLDER_1**. We explicitly teach it the business rules of cricket statistics:

- Batsman runs must sum **CODE_PLACEHOLDER_0** where **CODE_PLACEHOLDER_1** matches the player's name.
- Bowler wickets must only count deliveries where the dismissal type is a bowler wicket (**CODE_PLACEHOLDER_0**, **CODE_PLACEHOLDER_1**, etc.), excluding non-bowler dismissals like run outs.

Providing these rules directly in the system prompt prevents the LLM from writing incorrect queries.

Q10: How does the background seeding endpoint prevent API timeouts during large data loads?

Answer: Standard API requests must return a response quickly to avoid client timeouts. Seeding a database can take several minutes.

We solve this using FastAPI's built-in **CODE_PLACEHOLDER_0** runner:

```
@router.post("/stats/seed-database")
def seed_database(background_tasks: BackgroundTasks):
    background_tasks.add_task(load_match_data)
    return {"status": "success", "message": "Database seeding triggered..."}
```

FastAPI immediately returns a **CODE_PLACEHOLDER_0** response to the client, and then runs the seeding function asynchronously in the background without blocking the main event loop.

Q11: Explain the purpose of **CODE_PLACEHOLDER_0** in the root of the project.

Answer: **CODE_PLACEHOLDER_0** is the configuration file for **Pyright**, a fast static type checker for Python. It tells the type checker where our virtual environment is located (**CODE_PLACEHOLDER_1**) and defines search paths (**CODE_PLACEHOLDER_2**). This allows Pyright to resolve import paths correctly, providing real-time code completion, error detection, and type safety checks in our editor.

Q12: How would you run automated tests for this backend, and how is the database isolated?

Answer: We use **pytest** for testing. In **CODE_PLACEHOLDER_0**, we define a fixture that creates a temporary SQLite file **CODE_PLACEHOLDER_1** and runs **CODE_PLACEHOLDER_2** to build a clean schema for each test session. This ensures our production/development database is never affected by test writes, giving us safe and isolated testing.

Q13: What happens if the OpenAI API is down? How is the application affected?

Answer: The **CODE_PLACEHOLDER_0** route will catch the connection exception, print the error, and return an HTTP **CODE_PLACEHOLDER_1**. However, the rest of the application remains fully functional. The health check and REST endpoints (matches, scorecards, stats) do not rely on OpenAI and will continue to work normally.

Q14: How does the application handle stadium name variations (e.g., "Gaddafi Stadium" vs. "Gaddafi Stadium, Lahore")?

Answer: We solve this using two techniques:

1. **Fuzzy String Map:** We maintain a dictionary of common stadium aliases in **CODE_PLACEHOLDER_0** (**CODE_PLACEHOLDER_1**) to resolve queries to their canonical database names.
2. **Case-Insensitive SQL CODE_PLACEHOLDER_0:** We instruct the AI agent to always query text columns using case-insensitive SQL matching (e.g., **CODE_PLACEHOLDER_1**), ensuring matches succeed even with minor name variations.

Q15: What is the role of CODE_PLACEHOLDER_0 in the CODE_PLACEHOLDER_1 file?

Answer: It tells Python to include the **CODE_PLACEHOLDER_0** folder in its list of module search directories. This allows scripts (like our data pipeline or testing suite) to run from any folder while using clean absolute imports (e.g., **CODE_PLACEHOLDER_1**) without throwing module import errors.